ENGINEERING PLAYBOOK

Apache Spark / PySpark 3.x+

PySpark Optimization Techniques

Masterclass on Eliminating Network Shuffles, Resolving Data Skew, and Tuning Physical Execution Plans

---

Zero-Shuffle Joins

Skew Mitigation

AQE Engine Tuning

PERFORMANCE LANDSCAPE

The 3 Main PySpark Performance Bottlenecks

1. Network Shuffles

Wide transformations (`Exchange`) serialize and ship data across cluster nodes.

2. Straggler Data Skew

Uneven key distribution overloads a single executor while others sit idle.

3. Post-Join Explosions

Joining unaggregated raw tables multiplies row counts before grouping.

TECHNIQUE #1

Broadcast Hash Joins: Eliminating Shuffles

How It Works

Copies small dimension tables directly to worker nodes, replacing network shuffles with local in-memory lookups.

- **Automatic Threshold:** Default size is 10MB (`autoBroadcastJoinThreshold`).
- **Multi-Table Hints:** Broadcast multiple lookup tables in a single query.

Spark SQL & PySpark API — Zero Shuffle Exchange

```
-- Spark SQL Broadcast Hint
SELECT /*+ BROADCAST(s), BROADCAST(i) */
    t.txn_id, s.location, i.category, t.amount
FROM fact_transactions t
JOIN dim_stores s ON t.store_id = s.store_id
JOIN dim_items i  ON t.item_id = i.item_id;

# PySpark Broadcast DataFrame API
from pyspark.sql.functions import broadcast
df = fact.join(broadcast(stores), "store_id")
```

TECHNIQUE #2

Pre-Aggregation: Reducing Shuffle Data Volume

Wrong: Join First, Aggregate Later

Shuffles raw uncompressed event streams across the network prior to rollup operations.

```
activities.join(users, "user_id")
  .groupBy("tier", "action")
  .count()
```

Result: High Network I/O & Potential OOM

Right: Aggregate First, Join Summary

Compresses dataset locally on each partition prior to executing network shuffle joins.

```
summary = activities.groupBy("user_id", "action").count()
summary.join(users, "user_id")
  .groupBy("tier", "action").sum("count")
```

Result: Up to 90% Reduced Shuffle Traffic

TECHNIQUE #3

Salting Skewed Keys: Resolving Executor Bottlenecks

1 Salt Left Fact Table

Append random integer suffixes (`key_0`, `key_1`) to break up heavy keys.

2 Explode Dimension Table

Replicate dimension keys across matching salt ranges using cross joins.

3 Parallel Equal Distribution

Distributes single-node bottlenecks evenly across multiple worker nodes.

Key Takeaway: Eliminates 99%-done straggler tasks stuck on massive skewed keys.

TECHNIQUE #4

Two-Phase Aggregation: Fixing Post-Join Traps

2-Phase

Partial-Then-Final Rollup

Grouping by raw keys right after salting re-introduces skew shuffles. Preserve salt through Phase 1.

Phase 1

Group By Salted Key

Compute partial sums inside individual salt buckets without triggering key collisions.

Phase 2

Strip Salt & Final Reduction

Collapse pre-reduced salt bucket totals down to the original key with minimal data volume.

TECHNIQUE #5

Proactive Write-Time Optimizations

Spark Bucketing (`bucketBy`)

Pre-sorts and hashes data across fixed storage buckets on disk during table ingestion.

- **Zero-Shuffle Joins:** Pre-aligned bucket files merge locally.
- **Best For:** Frequent batch joins on fixed primary keys.

```
df.write.bucketBy(16, "user_id").saveAsTable("...")
```

Quarantine Pattern

Isolates toxic or default keys (`NULL`, `tenant_0`) into separate storage feeds.

- **Clean Main Table:** 99% of data remains uniform.
- **Isolated Handling:** Toxic keys processed independently.

```
toxic_df = df.filter(col("id").isNull())
```

TECHNIQUE #6

Adaptive Query Execution (AQE) Skew Joins

Automated Dynamic Remapping

Spark 3.x+ inspects runtime shuffle statistics to split skewed partitions automatically without code changes.

spark.sql.adaptive.enabled	true
spark.sql.adaptive.skewJoin.enabled	true
skewedPartitionFactor	5 (Default)

Physical Plan Verification

Run `explain(True)` to confirm runtime plan re-optimization:

```
AdaptiveSparkPlan isFinalPlan=true
+- AdaptiveSkewedJoin [key#74L]
   :- SortMergeJoin [key#74L]
   +- Exchange (Coalesced & Split)
```

TECHNIQUE #7

Manual Map-Side Joins via Broadcast Variables

Bypassing Catalyst Join Overhead

Collects small dimension tables into Python dictionaries on the driver and distributes them via `sparkContext.broadcast()`.

> **O(1) Memory Lookups**
> Executes in-memory lookups inside `mapPartitions()` with zero engine overhead.

```python
# 1. Collect small table to driver dictionary
dim_dict = {r["id"]: r["name"] for r in dim.collect()}

# 2. Broadcast across cluster
b_dim = spark.sparkContext.broadcast(dim_dict)

# 3. Partition-level lookup
def lookup_fn(iterator):
    table = b_dim.value
    for row in iterator:
        yield (row.id, table.get(row.id, "N/A"))

df.rdd.mapPartitions(lookup_fn)
```

ARCHITECTURE SUMMARY

Optimization Decision Matrix

Small Dimension Table?

Use **Broadcast Join Hints** or **Map-Side Broadcast Variables**.

Severe Data Skew?

Apply **Salting** or enable **AQE Skew Join Remapping**.

Heavy Aggregations?

Execute **Pre-Aggregation CTEs** or **Two-Phase Salted Rollups**.

Questions & Technical Discussion